



UNIVERSIDAD TECNOLÓGICA DE PANAMÁ

FACULTAD DE INGENIERÍA DE SISTEMAS COMPUTACIONALES

DEPARTAMENTO DE COMPUTACIÓN Y SIMULACIÓN DE SISTEMAS

LICENCIATURA EN INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

Asignatura:

Sistemas Operativos

Asignación:

Laboratorio 4: Incidente de hiperpaginación (thrashing) y swap, informe RCA del servicio en el puerto 5002

Profesor:

Prof. Isacio Tamayo

Estudiante:

James Tack

Grupo:

1IL143

Fecha de entrega:

2 de octubre de 2026

2do Semestre 2026

De qué se trata el laboratorio

El laboratorio plantea un incidente de producción. Un microservicio Flask (`app.py`, puerto 5002) tiene un endpoint, `/api/v1/analytics`, que pide 180 MB de memoria para procesar datos. El contenedor se despliega con 128 MB de RAM y 384 MB de swap. El servicio no se cae y sigue devolviendo `HTTP 200`, pero cada llamada tarda decenas de segundos en vez de una fracción de segundo. Eso es hiperpaginación (thrashing). La regla del laboratorio es que `app.py` no se modifica; el arreglo va en el contenedor y en el sistema operativo.

Trabajé en la máquina virtual del curso (vm-sistemas-operativos, con Ubuntu 18.04, kernel 4.15.0-213, 2 CPU y 2 GB de RAM, Docker 24.0.2 con cgroups v2). La guía pide cuatro evidencias de pantalla (E1 a E4) y un documento técnico. Cada comando lo corrí uno a la vez en una pestaña de Windows Terminal conectada por SSH a la VM, y las capturas son de esas pestañas.

Conceptos que usa el informe

El swap es espacio en disco que el kernel usa como extensión de la RAM. Cuando un proceso toca una página que está en swap, ocurre un page fault y el kernel la trae de vuelta del disco, que es miles de veces más lento que la RAM. Si el conjunto de páginas que el proceso necesita no cabe en memoria y las reescribe una y otra vez, el sistema pasa más tiempo moviendo páginas que ejecutando el programa. Eso es thrashing. En `vmstat` se ve con `si` (swap in) y `so` (swap out) sostenidos y `wa` (espera de E/S) alto.

Docker aplica los límites con cgroups v2. Cada contenedor tiene una carpeta en `/sys/fs/cgroup` con archivos que se leen y se escriben como cualquier otro, entre ellos `memory.max` (tope de RAM), `memory.swap.max` (tope de swap), `memory.current` (lo usado ahora), `memory.events` (contadores de `max`, `oom` y `oom_kill`) y `cpu.max`. El flag `--memory-swap` de Docker es el total de RAM más swap, no el swap solo. Con `--memory=128m --memory-swap=512m`, el swap real es 512 − 128 = 384 MB.

La diferencia con un OOM Killer es que aquí el kernel nunca necesita matar a nadie, porque tiene a dónde mandar las páginas. Con `memory.swap.max = 0`, ese exceso de memoria terminaría en `oom_kill`.

Preparación del entorno

Paso 1. Encender la máquina virtual

La VM estaba apagada. La encendí con `vagrant up` desde la carpeta del laboratorio 1 y comprobé con `vagrant status` y `VBoxManage showvminfo` que quedó en estado `running`, con 2 CPU y 2048 MB.

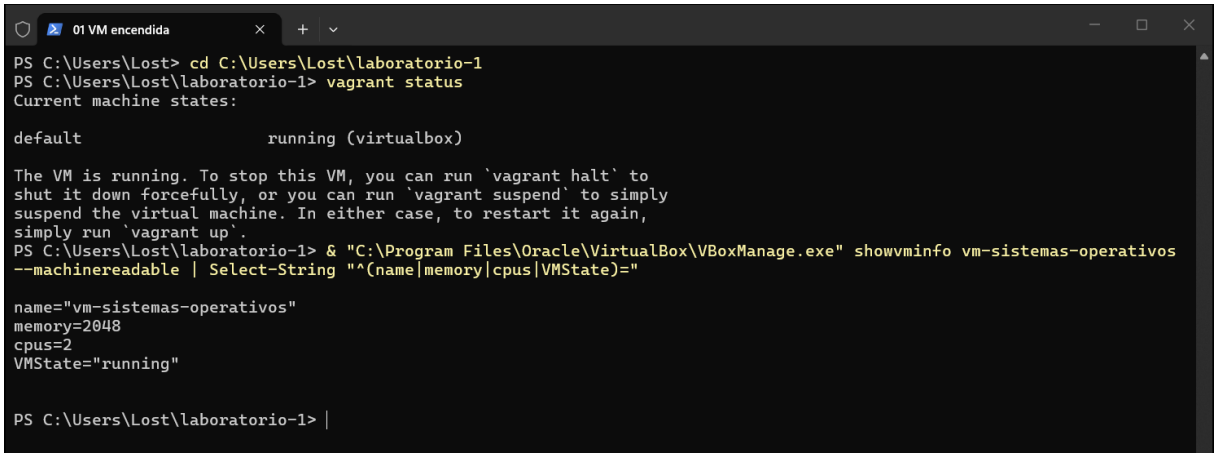


Figura 1. vagrant status y VBoxManage confirman que vm-sistemas-operativos está encendida (2 CPU, 2048 MB).

Paso 2. Copiar los archivos a la VM

El laboratorio trae `app.py` y el `Dockerfile`. Los copié con `scp` y la llave privada que genera Vagrant a `~/laboratorios/lab4` dentro de la VM.

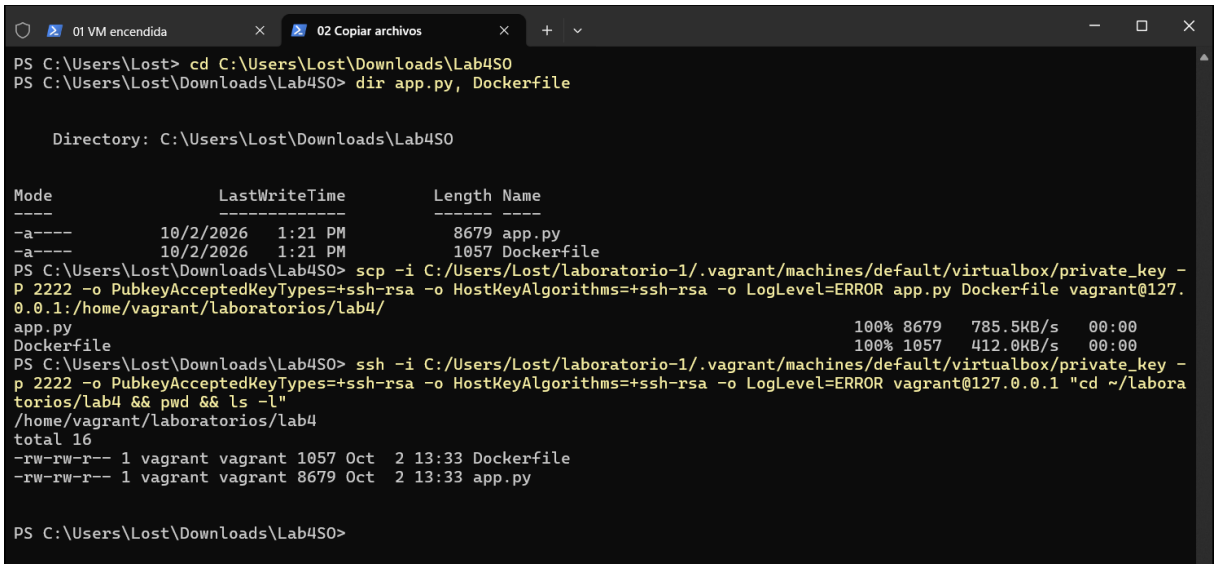


Figura 2. app.py y Dockerfile en Windows, copia con scp y listado en la VM.

Paso 3. Revisar los prerrequisitos

El documento de prerrequisitos pide Docker activo, los puertos 5002 y 5003 libres, al menos 600 MB de RAM libre y swap del host habilitado. Docker responde con Cgroup Version 2 (la VM ya quedó en v2 desde la tarea 3, con los parámetros `systemd.unified_cgroup_hierarchy=1 swapaccount=1` en el arranque) y los puertos están libres. Pero `swapon --show` no devolvió nada y `free -m` marca `Swap: 0`. La guía dice que sin swap el incidente no se puede reproducir.

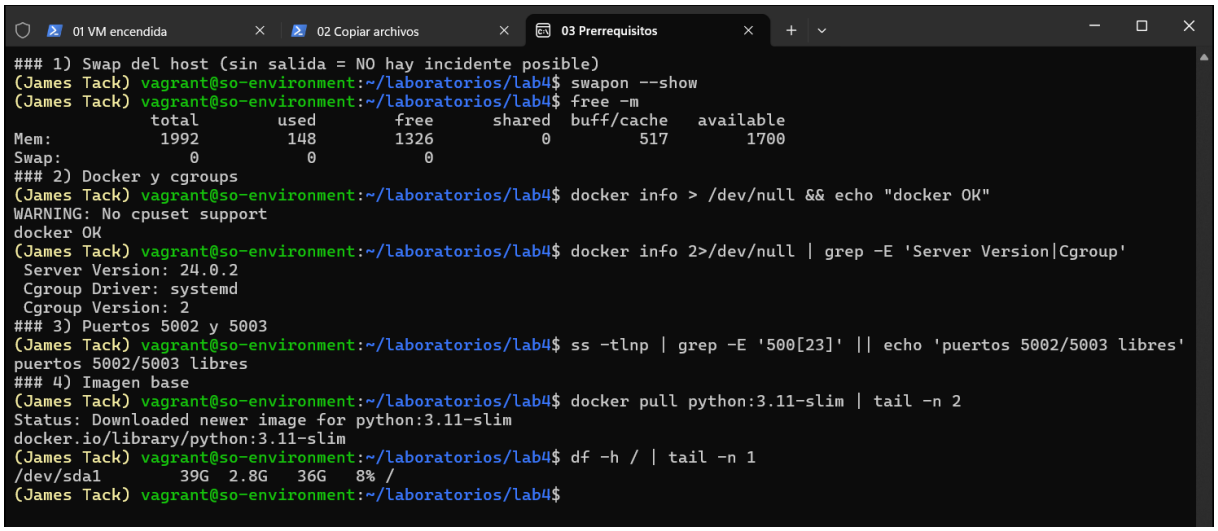


Figura 3. Docker y los puertos están bien, pero la VM no tiene swap.

Paso 4. Crear el swap que falta

En lugar de avisar al docente, creé un archivo de swap de 2 GB (`fallocate`, `mkswap`, `swapon`) y lo agregué a `/etc/fstab` para que sobreviva a un reinicio. El disco de la VM tiene 36 GB libres. Después de `swapon`, `swapon --show` lista `/swapfile` de 2 GB y `free -m` muestra `Swap: 2047`. El `vm.swappiness` por defecto es 60.

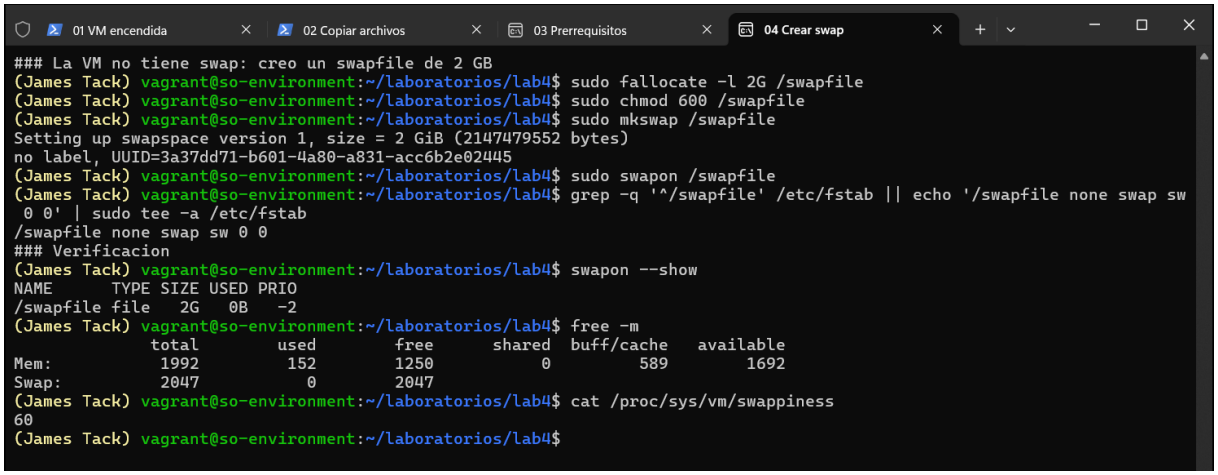


Figura 4. Creación del swapfile de 2 GB y verificación con swapon y free.

Paso 5. Revisar el Dockerfile y app.py

La imagen parte de `python:3.11-slim`, instala Flask y arranca `python app.py`, que queda como PID 1 del contenedor. En `app.py`, la ruta `/api/v1/analytics` reserva un `bytearray` de 180 MB y luego recorre el buffer tres veces escribiendo un byte cada 4096, o sea una vez por página de 4 KB. Esa escritura obliga al kernel a materializar cada página y a tocarla tres veces. Con un tope de 128 MB, el buffer no cabe y las páginas se van y vuelven del swap.

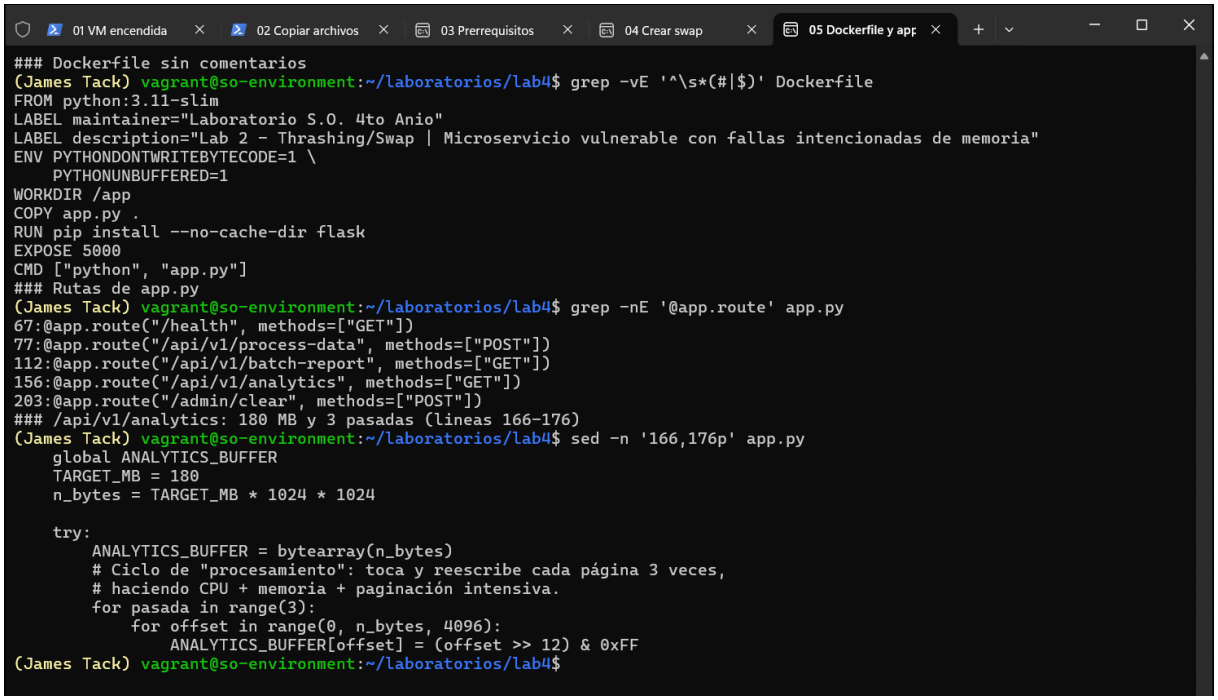


Figura 5. Dockerfile sin comentarios, rutas de app.py y el ciclo de tres pasadas de /analytics.

Paso 6. Construir la imagen

Construí la imagen con `docker build -t app-reto2-thrashing:v1 .`. Tardó unos 10 segundos y la imagen quedó de 138 MB.

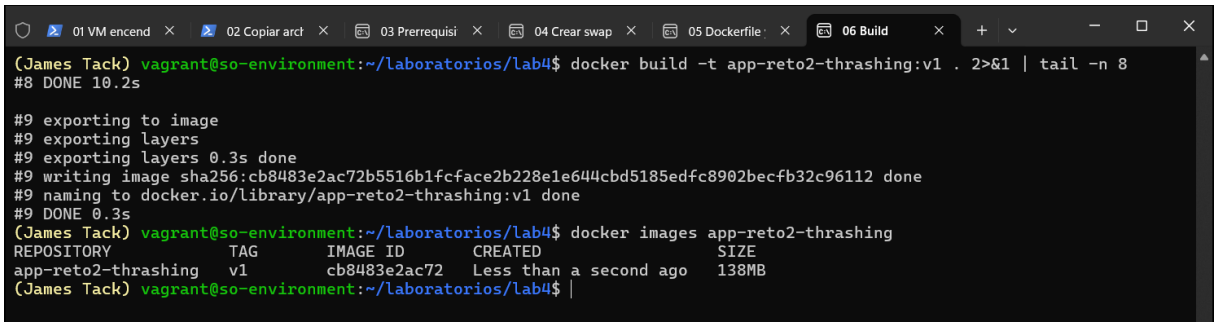


Figura 6. docker build termina y app-reto2-thrashing:v1 aparece en docker images.

Paso 7. Levantar el contenedor con los límites del incidente

```
docker run -d --name reto2_app --restart no -p 5002:5000 \
  --memory="128m" --memory-swap="512m" --cpus="1" app-reto2-thrashing:v1
```

`--restart no` evita que Docker lo reinicie solo, para ver si muere. `/health` responde `{"message":"Servicio vivo","pid":1,"status":"ok"}`.

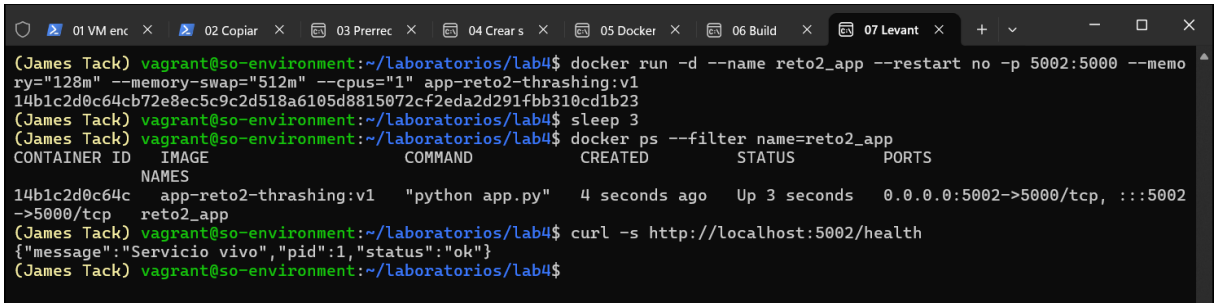


Figura 7. Contenedor reto2_app en ejecución y respuesta de /health.

Evidencias del incidente

Evidencia 1. Los límites con los que se levantó el servicio

Este bloque separa dos fuentes, lo que Docker dice que configuró (`docker inspect`) y lo que el kernel aplica de verdad (los archivos del cgroup, leídos dentro del contenedor con `docker exec`). Primero compruebo el contexto. `hostname` devuelve el ID del contenedor y `/proc/1/comm` devuelve `python`, así que los comandos corren dentro.

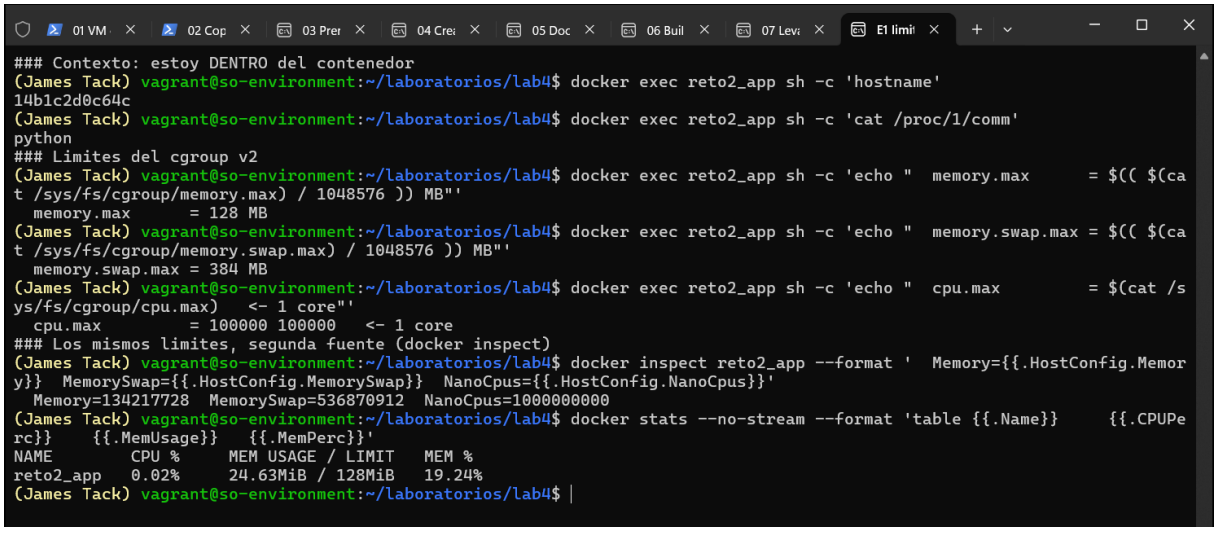


Figura 8. E1: memory.max = 128 MB, memory.swap.max = 384 MB, cpu.max = 100000 100000 (1 núcleo); docker inspect y docker stats coinciden.

Límite	Valor en el cgroup	Valor en docker inspect
RAM (memory.max)	128 MB	Memory = 134217728
Swap (memory.swap.max)	384 MB	MemorySwap = 536870912 (total RAM + swap)
CPU (cpu.max)	100000 100000 = 1 núcleo	NanoCpus = 1000000000

El swap sale de la resta, porque `--memory-swap=512m` es el total, y 512 − 128 = 384 MB. En reposo el contenedor usa 24,63 MiB de 128 MiB (19,24 %). Esta es la línea base del incidente.

Evidencia 2. El incidente: el servicio se degrada

El bloque llama tres veces a `/api/v1/analytics` midiendo el tiempo con `curl -w`, captura `vmstat 1` durante una llamada, lee `VmRSS` y `VmSwap` del proceso, compara `memory.current` con `memory.max` y lee `memory.events`. El `sed` que se ve en el `vmstat` descarta la línea de promedio desde el arranque, que no sirve.

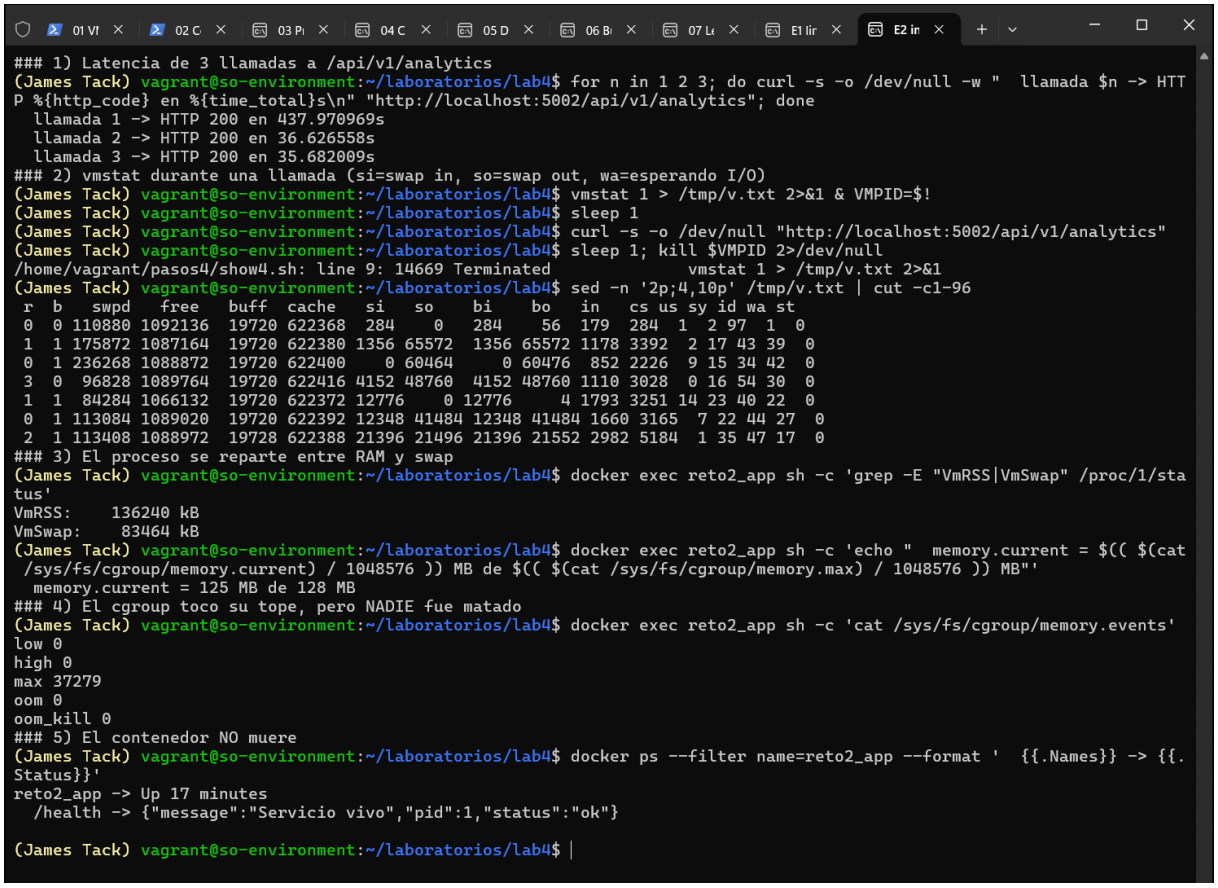


Figura 9. E2: tres llamadas con HTTP 200 pero muy lentas, vmstat con swap in/out sostenido, el proceso repartido entre RAM y swap, y oom_kill 0.

De esta captura salen cuatro datos.

- Las tres llamadas devolvieron `HTTP 200` en 437,97 s, 36,63 s y 35,68 s. La respuesta normal es de unos 0,1 s (la medí después, en E3). Ese es el síntoma.
- En `vmstat`, `so` llega a 65 572 kB/s y `si` a 21 396 kB/s, con `wa` entre 17 y 42 %. El swap entra y sale al mismo tiempo, que es la firma de la hiperpaginación.
- Con `VmRSS` de 136 240 kB y `VmSwap` de 83 464 kB, el proceso queda repartido entre RAM y swap. Unos 81 MB del proceso están en disco. `memory.current` marca 125 MB de 128 MB, o sea la RAM del contenedor llena.
- Tampoco es un OOM. `memory.events` marca `max 37279` (el cgroup tocó su tope muchas veces) pero `oom 0` y `oom_kill 0`. Nadie fue matado, y el contenedor sigue `Up` con `/health` respondiendo.

Corrí este bloque cuatro veces, dos de ellas por fallos míos al capturar (una salida se salió de la pantalla y en otra el bucle de `curl` estaba mal escrito). Las latencias no fueron iguales. La corrida anterior a la captura que uso dio 26,6 s, 26,2 s y 235,4 s, y la captura, 438 s, 36,6 s y 35,7 s. La guía menciona 7 a 18 s y avisa que depende de la máquina. Aquí el swap está en un archivo sobre el disco virtual de VirtualBox, que además comparte disco con Windows, y eso explica que el tiempo varíe tanto. No medí qué parte de la variación viene del host, así que lo dejo como explicación probable y no comprobada. En todas las corridas el patrón se repite, con decenas de segundos o más, `HTTP 200` y `oom_kill 0`.

Preguntas guía del diagnóstico

1. ¿Por qué hay 384 MB de swap si el flag dice 512m? Porque `--memory-swap` es el total de RAM más swap. 512 − 128 = 384 MB, y es lo que se lee en `memory.swap.max`.

2. ¿Por qué el proceso no muere si no le entra la memoria? Porque hay swap. El kernel no necesita matarlo y expulsa páginas a disco. Si `memory.swap.max` fuera 0, el exceso terminaría en OOM Killer. Lo comprobé sin querer más adelante (ver el hallazgo sobre la concurrencia).

3. ¿Qué hace que sea thrashing y no solo memoria baja? El job reescribe cada página tres veces. Cada reescritura de una página que está en swap provoca un page fault y una lectura de disco (`si` alto), mientras el kernel expulsa otra página para hacer sitio (`so` alto). Con `si`, `so` y `wa` altos de forma sostenida, el tiempo se va en mover páginas y no en trabajo útil.

4. Triangulación con cuatro fuentes independientes. Desde el orquestador, `docker inspect` y `memory.max` dicen 128 MB (E1). Desde el kernel, `vmstat` con `si`, `so` y `wa` altos (E2). Desde el proceso, `VmRSS` de 136 MB y `VmSwap` de 83 MB (E2). Desde el cgroup, `memory.events` con `max` alto y `oom_kill 0` (E2).

Mitigación: liberar sin reiniciar

El servicio tiene un endpoint administrativo, `POST /admin/clear`, que suelta los buffers globales y fuerza el recolector de basura. Después de llamarlo, `VmRSS` quedó en 30 316 kB y `VmSwap` en 5 064 kB (en E2 eran 136 240 y 83 464). Pero la llamada siguiente a `/analytics` volvió a tardar 36,06 s, y `VmRSS` y `VmSwap` regresaron a 118 660 kB y 101 044 kB.

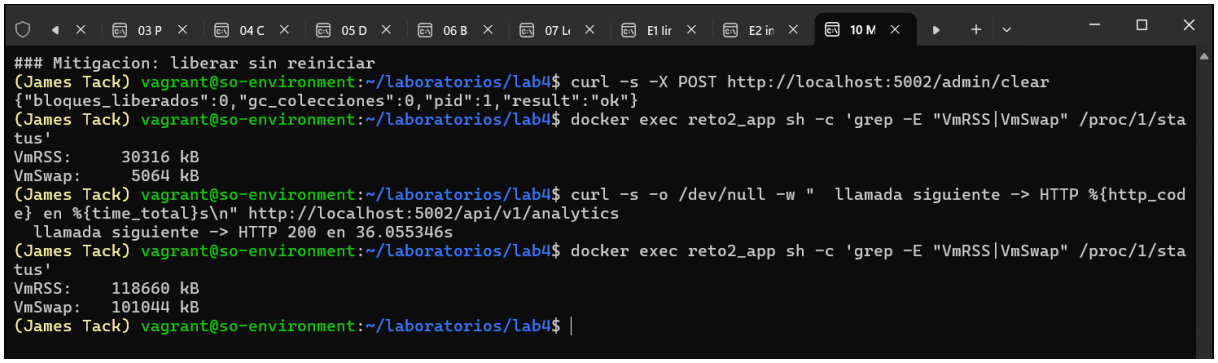


Figura 10. /admin/clear libera memoria, pero la siguiente llamada reproduce el incidente.

Esto es mitigación, no solución. Recupera el servicio un rato, pero no toca los límites, así que la próxima petición vuelve a pedir 180 MB contra 128 MB de RAM.

Solución a nivel de sistema operativo

La causa raíz es de dimensionado. El job necesita en el peor momento unos 210 MB (el buffer de 180 MB más lo que ya ocupa el intérprete y Flask) y el contenedor tiene 128 MB. Hay dos correcciones. Una es subir la RAM para que el job entre entero y la otra quitar el swap del contenedor, para que una falta de memoria futura avise con un OOM en vez de degradar el servicio en silencio. No toqué `app.py`.

Opción B primero: escribir directo en el cgroup del host

Antes de usar Docker probé la vía de bajo nivel. Saqué el ID del contenedor con `docker inspect`, armé la ruta `/sys/fs/cgroup/system.slice/docker-<id>.scope` y escribí con `sudo sh -c "echo ... > archivo"`. `memory.max` aceptó 536870912 (512 MB). Pero escribir 0 en `memory.swap.max` falló con `I/O error` y el valor se quedó en 402653184. El kernel 4.15 no deja bajar `memory.swap.max` por debajo del swap que el cgroup ya usa, y en ese momento usaba 100 MB. Además, `docker inspect` seguía mostrando `Memory=134217728`. El kernel cambió, pero la configuración de Docker no.

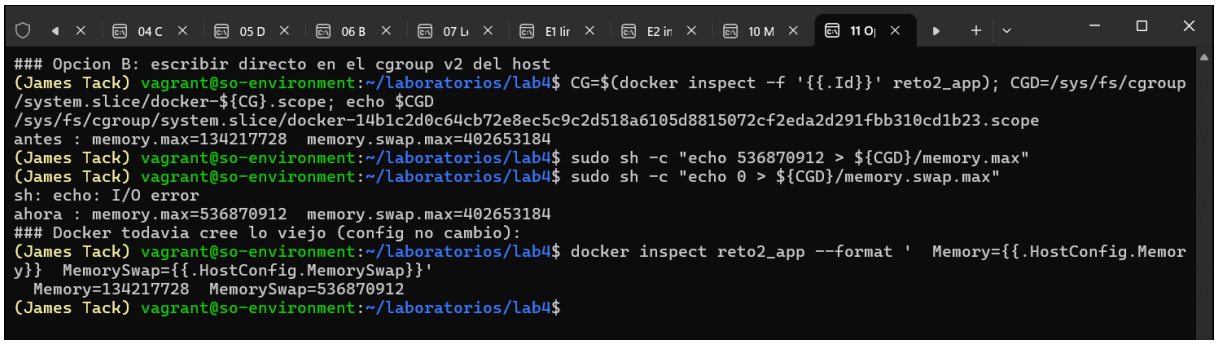


Figura 11. memory.max sube a 512 MB; memory.swap.max = 0 es rechazado con I/O error; Docker sigue creyendo 128 MB.

Para poder poner el swap en 0 primero hay que vaciarlo. Con `memory.max` ya en 512 MB, una llamada a `/analytics` tardó 1,65 s (la RAM ya alcanza). Luego corrí `sudo swapoff -a && sudo swapon -a`, que obliga al kernel a traer todo el swap de vuelta a RAM y lo reactiva vacío. Con `memory.swap.current` en 0 MB, la escritura de 0 en `memory.swap.max` sí funcionó (`rc=0`).

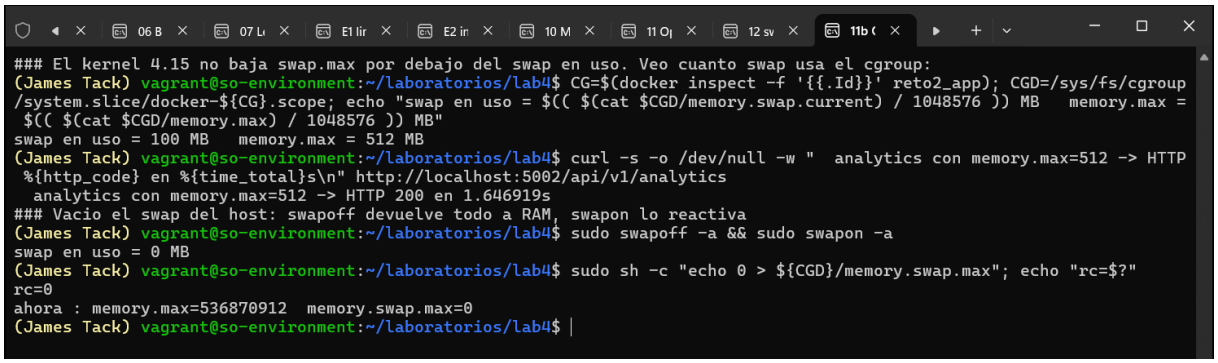


Figura 12. Con el swap del cgroup vaciado, memory.swap.max = 0 se acepta.

Complemento: bajar vm.swappiness

El host tenía `vm.swappiness = 60`, que hace al kernel dispuesto a mandar páginas a disco para liberar RAM. Lo bajé a 10 con `sudo sysctl -w vm.swappiness=10`. El cambio no persiste tras un reinicio (para eso habría que escribirlo en `/etc/sysctl.d/`). La guía también pide mirar `/proc/pressure/memory`, pero ese archivo no existe en el kernel 4.15; la información de presión (PSI) llegó en el 4.20.

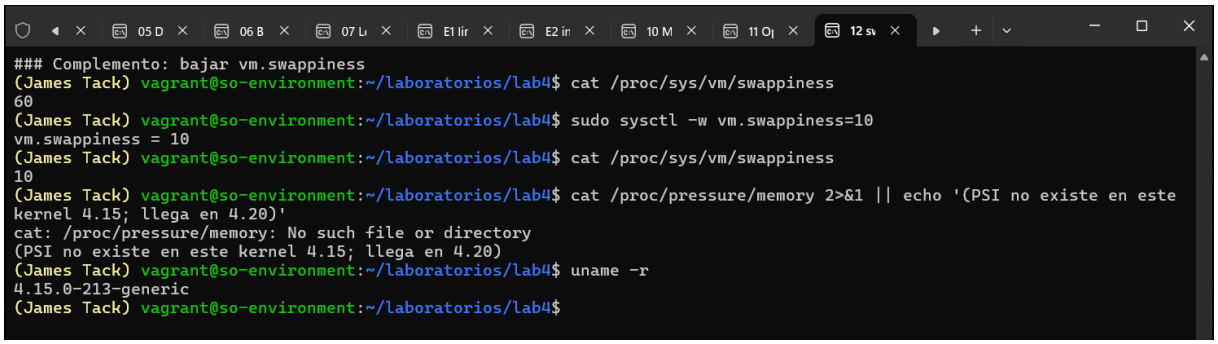


Figura 13. vm.swappiness pasa de 60 a 10; /proc/pressure/memory no existe en el kernel 4.15.

Opción A: docker update (la que queda guardada en Docker)

Para que el cambio quede en la configuración del contenedor y no solo en el kernel, usé `docker update --memory 512m --memory-swap 512m reto2_app`. Como `--memory-swap` es igual a `--memory`, el swap queda en 0. Se aplica en caliente, sin recrear el contenedor. Como ya había hecho la Opción B, los valores del cgroup no cambiaron al correrlo; lo que cambió es que ahora `docker inspect` coincide con el kernel (`Memory=536870912 MemorySwap=536870912`).

Evidencia 3. El cambio aplicado y verificado

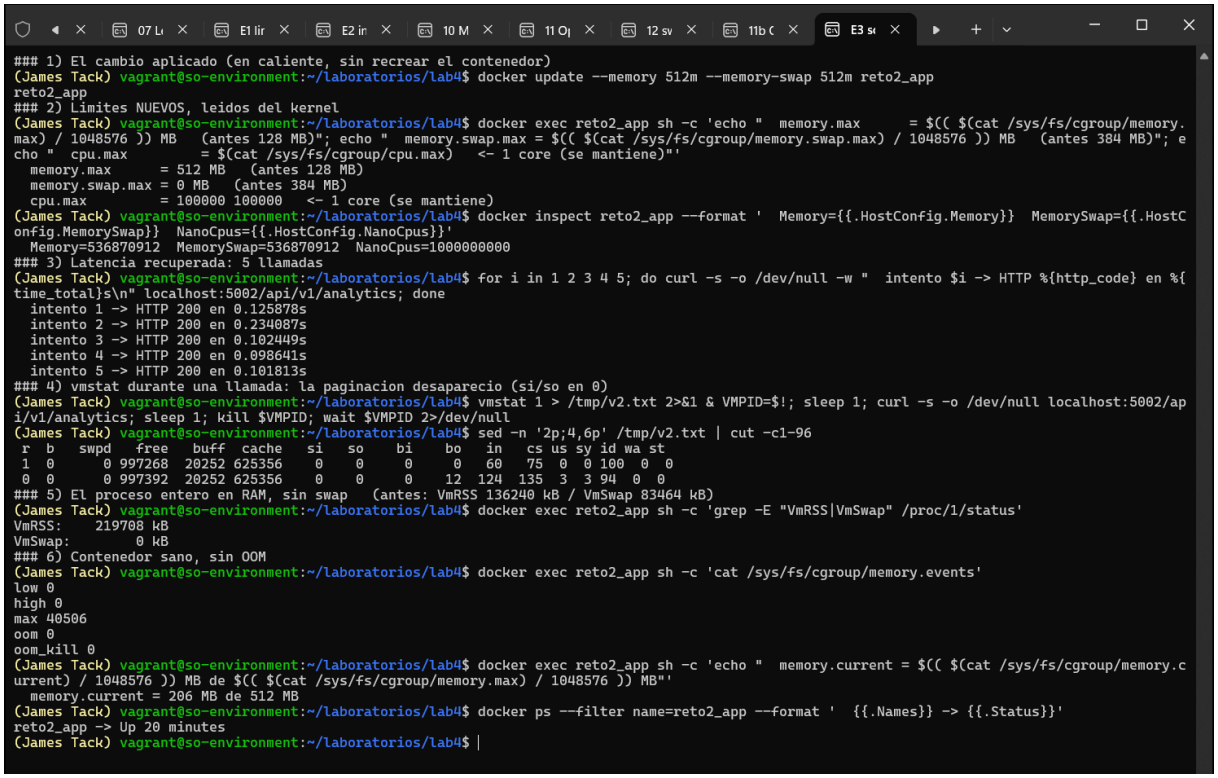


Figura 14. E3: límites nuevos (512 MB, swap 0), cinco llamadas de ~0,1 s, vmstat con si/so en 0, VmSwap en 0 kB y oom_kill 0.

- Los límites quedaron en 512 MB de `memory.max` (antes 128), 0 MB de `memory.swap.max` (antes 384) y `cpu.max` sin cambio.
- Las cinco llamadas dieron `HTTP 200` en 0,126 s, 0,234 s, 0,102 s, 0,099 s y 0,102 s. De 36 a 438 s a unos 0,1 s, o sea entre unas 360 y más de 4000 veces más rápido, según la corrida del incidente que se compare.
- Durante una llamada, `si` y `so` se quedan en 0.
- El proceso entero está en RAM, con `VmRSS` de 219 708 kB y `VmSwap` de 0 kB. `memory.current` 206 MB de 512 MB.
- `oom_kill 0`. El contador `max` de `memory.events` (40506) es un acumulado desde que arrancó el cgroup y no baja nunca; lo que demuestra el arreglo es la latencia, `si/so` en 0 y `VmSwap` en 0, no ese contador.

Hallazgo: 512 MB no alcanza con peticiones simultáneas

Para la evidencia 4 la guía pide dejar el servicio trabajando con un bucle de `curl`. Lancé el bucle y, además, hice una llamada suelta de `curl` para comprobar la latencia bajo carga. Esa llamada devolvió `HTTP 000` (conexión cortada) tras 1,87 s. El contenedor había muerto.

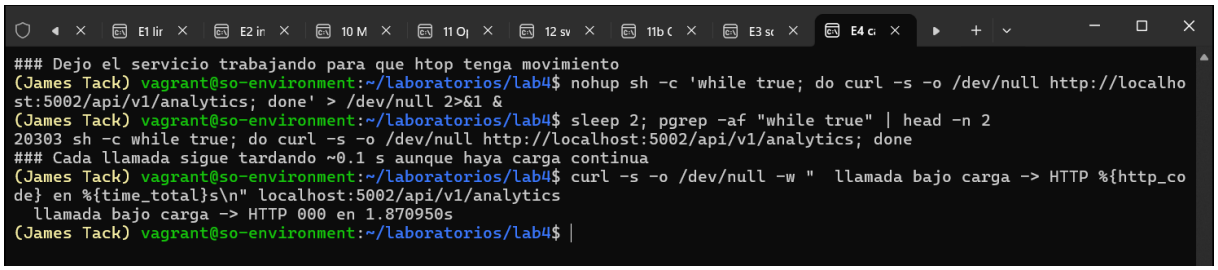


Figura 15. Una llamada extra mientras corría el bucle: HTTP 000, la conexión se cortó.

`docker inspect` dice `ExitCode=137` y `OOMKilled=true`, y `dmesg` registra `Memory cgroup out of memory: Kill process 5818 (python)` con `anon-rss:521348kB`. Es decir, esta vez sí hubo OOM Killer, porque con `memory.swap.max = 0` el cgroup no tiene dónde mandar páginas.

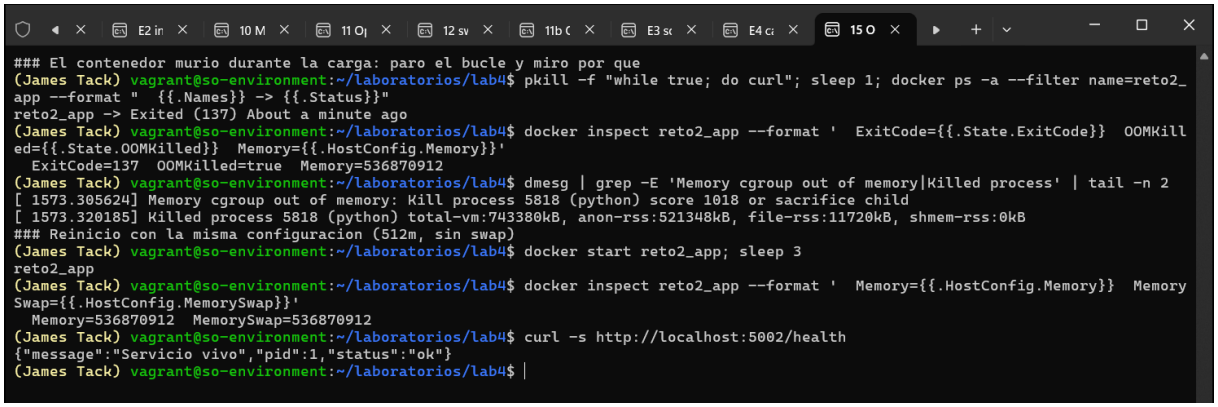


Figura 16. El contenedor corregido murió por OOM con dos peticiones simultáneas; se reinicia con docker start y mantiene 512 MB sin swap.

Mi explicación, a partir de `app.py`, es que el endpoint asigna el buffer nuevo (`ANALYTICS_BUFFER = bytearray(...)`) antes de soltar el anterior, así que una petición sola llega a unos 180 + 180 + 30 = 390 MB y cabe en 512. Flask atiende cada petición en un hilo. Con dos peticiones a la vez, hay unos tres buffers de 180 MB vivos a la vez y el total pasa de 512 MB. El `anon-rss` de 521 MB en `dmesg` encaja con eso. Es una explicación por el código y por la cifra, no la verifiqué lanzando exactamente dos peticiones a propósito.

Esto implica que los 512 MB de la guía resuelven el incidente para una petición a la vez, que es lo que prueba la guía, pero no dejan margen para concurrencia. Para producción habría que dimensionar con la regla de más o menos (peticiones simultáneas + 1) × 180 MB + 30 MB, o limitar la concurrencia del servicio (un solo worker, o un servidor WSGI con cola). Mantuve los 512 MB porque son los que pide la guía y los que verifica E3, y el bucle de E4 hace una petición a la vez.

Evidencia 4. htop y docker stats con los límites corregidos

Reinicié el contenedor con `docker start` (conserva 512 MB y swap 0) y dejé corriendo un bucle de una sola petición a la vez.

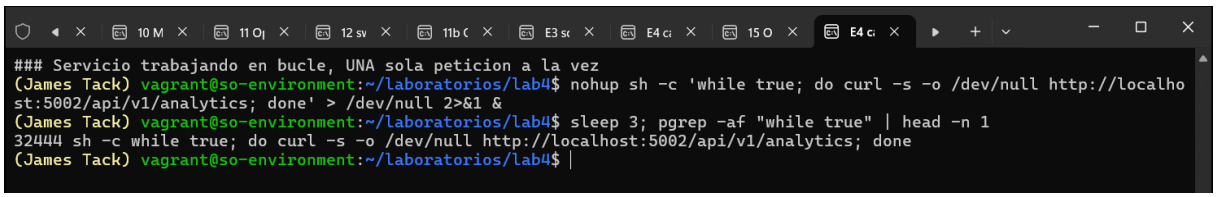


Figura 17. Bucle de curl en segundo plano, una petición a la vez.

En otra pestaña abrí `htop -s PERCENT_MEM`, filtré con F4 por `app.py` y activé la vista de árbol con F5. Se ve el proceso `python app.py` con 290 MB residentes y 104 % de CPU (un núcleo, el tope de `--cpus=1`). El medidor Swp del host marca `0K/2.00G` y Mem `473M/1.95G`. Es decir, el proceso trabaja a tope de CPU y sin tocar el disco.

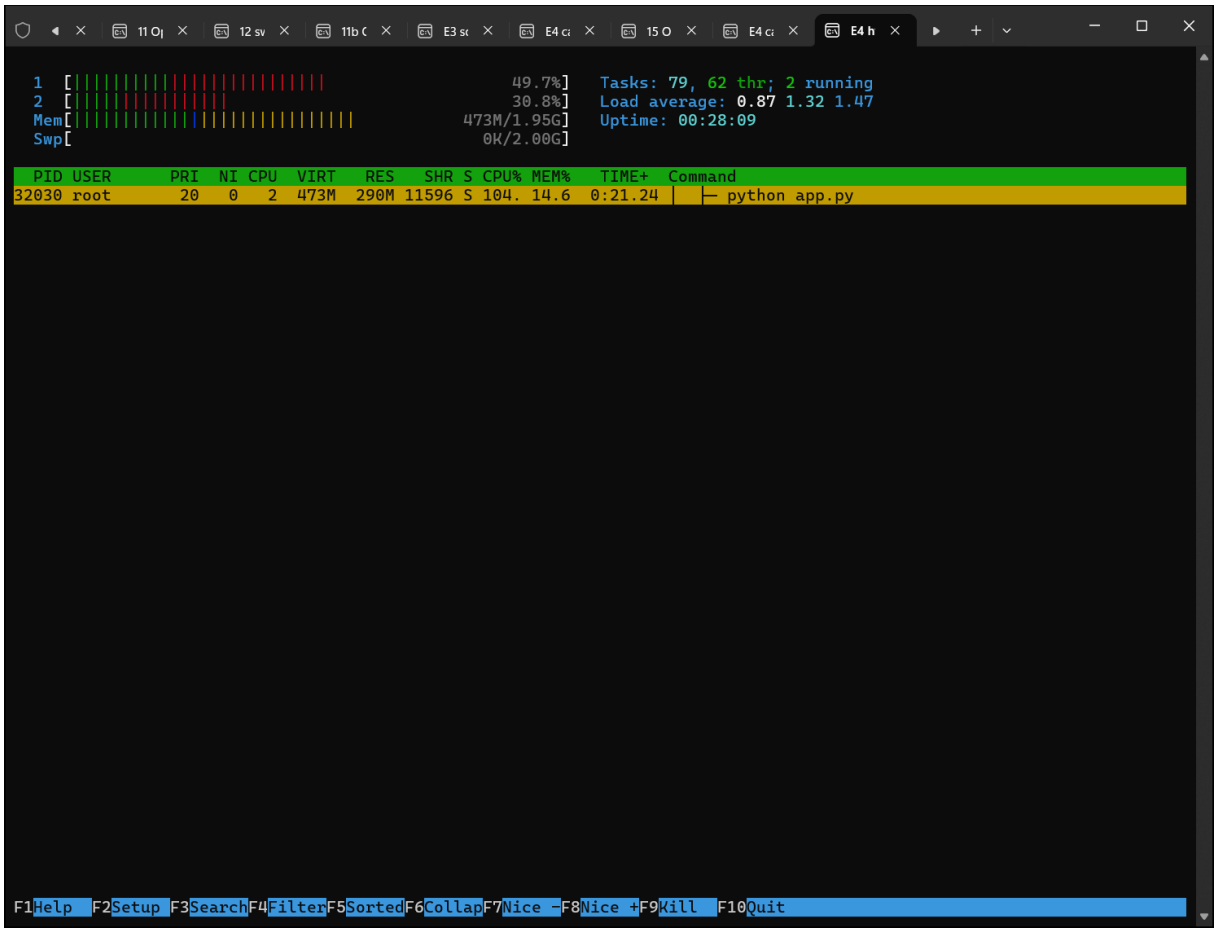


Figura 18. E4: htop filtrado por app.py, en árbol y ordenado por memoria. Swp del host en 0K.

htop no muestra los límites del cgroup, así que el límite se demuestra con `docker stats`, que marca `296,5MiB / 512MiB` (57,92 %) y 100,52 % de CPU. Además `memory.swap.max` leído dentro del contenedor vale 0.

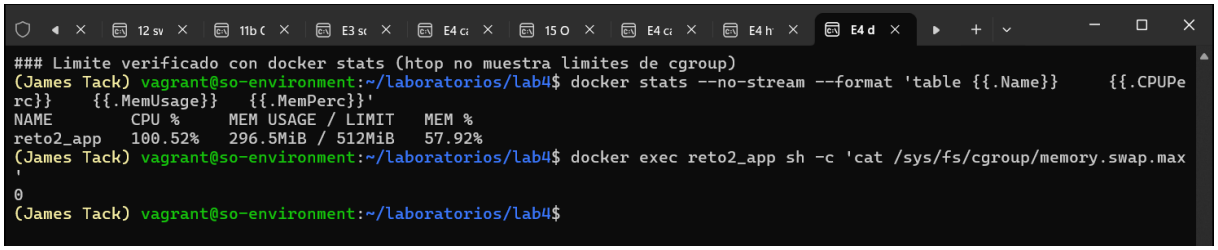


Figura 19. E4: docker stats con el límite de 512 MiB y memory.swap.max = 0.

Detuve el bucle y dejé el contenedor en su configuración final.

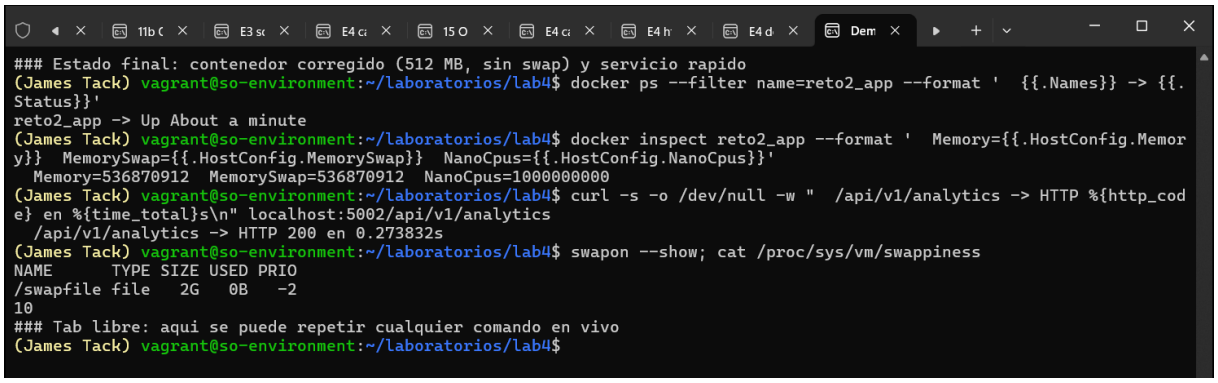


Figura 20. Estado final: reto2_app Up con 512 MB / swap 0, /analytics en ~0,1 s, swap de 2 GB activo y swappiness 10.

Informe RCA

1. Resumen ejecutivo

El servicio de analíticas respondía con `HTTP 200` pero tardaba entre 26 y 438 s por llamada, contra 0,1 s cuando está sano. La causa es que el contenedor tenía 128 MB de RAM y 384 MB de swap, y el job necesita unos 210 MB. El exceso se iba a swap y el job reescribe cada página tres veces, así que el kernel pasó el tiempo paginando (thrashing). No hubo OOM. Se corrigió subiendo la RAM a 512 MB y quitando el swap del contenedor, sin modificar `app.py`.

2. Síntoma reportado y síntoma real

Lo que se ve desde fuera	Lo que pasa por dentro
El contenedor está Up, /health responde, no hay errores 5xx.	memory.current pegado al tope (125 de 128 MB); memory.events con max en decenas de miles.
Las llamadas a /analytics tardan decenas de segundos.	vmstat con si/so sostenidos y wa de 17 a 42 %; VmSwap de 83 a 101 MB.
Nada en los logs de la aplicación.	oom_kill 0: el kernel no mató a nadie, solo movió páginas a disco.

3. Timeline

Hora (VM)	Evento
13:33	app.py y Dockerfile copiados a ~/laboratorios/lab4.
13:3x	Se detecta que la VM no tiene swap; se crea /swapfile de 2 GB y se activa.

Hora (VM)	Evento
13:35	Se levanta reto2_app con 128 MB de RAM y 384 MB de swap (E1).
13:3x–13:4x	Varias corridas de E2: HTTP 200 con latencias de decenas a cientos de segundos, oom_kill 0.
13:4x	/admin/clear libera memoria; la siguiente llamada tarda 36 s y vuelve a 100 MB de swap.
13:5x	memory.max a 512 MB por el cgroup; vaciado del swap; memory.swap.max = 0; docker update; E3 con 0,1 s.
13:56	OOM Killer mata el contenedor al hacer dos peticiones simultáneas con 512 MB y sin swap (exit 137).
13:57	docker start con la misma configuración; bucle de una petición; E4 (htop y docker stats).

Las horas con "x" son aproximadas porque no anoté la hora exacta de cada paso. Las de 13:33, 13:35 y 13:56 salen de la fecha del archivo, `docker inspect` y `dmesg`.

4. Análisis de causa raíz

Cinco porqués. (1) ¿Por qué el servicio tarda decenas de segundos? Porque casi cada acceso a memoria del job es un page fault contra el swap. (2) ¿Por qué usa swap? Porque el job toca unos 210 MB y el cgroup tiene 128 MB de RAM. (3) ¿Por qué no muere? Porque `memory.swap.max` es 384 MB y el kernel prefiere paginar antes que matar. (4) ¿Por qué hay 384 MB de swap? Porque `--memory-swap=512m` es el total y se leyó como si fuera solo swap. (5) ¿Por qué se desplegó así? Porque el límite se eligió sin medir el pico real del job y no hay alerta de latencia ni de swap que avise antes.

La causa raíz es el dimensionado, porque el límite de memoria es menor que el pico del job. El swap no es la causa; es lo que convierte un límite pequeño en lentitud en vez de en un error visible.

5. Mitigación y solución

Acción	Tipo	Resultado
POST /admin/clear	Mitigación	Libera ~100 MB, pero la siguiente llamada reproduce el incidente (36 s).
memory.max = 512 MB en el cgroup (Opción B)	Solución en el kernel	La llamada baja a 1,65 s; Docker no se entera.
swapoff -a / swapon -a y memory.swap.max = 0	Solución en el kernel	El kernel 4.15 exige vaciar el swap antes de bajar el tope.
docker update --memory 512m --memory-swap 512m	Solución persistente	docker inspect coincide con el kernel; 0,1 s por llamada, VmSwap 0.
vm.swappiness = 10	Complemento	El kernel prefiere caché antes que paginar (no persiste).

6. Prevención

- Medir el pico de memoria del job antes de fijar el límite, y dejar margen para peticiones simultáneas.
- Desplegar los servicios sin swap (`--memory-swap` igual a `--memory`). Un OOM se ve y se puede alertar; el thrashing pasa desapercibido.
- Alertar por latencia y por `memory.events` (`max` creciendo), no solo por si el contenedor está Up.

- Limitar la concurrencia de `/analytics`, o arreglar la causa en el código (liberar el buffer anterior antes de asignar el nuevo). Esto último queda fuera del laboratorio porque `app.py` no se puede modificar.

Tabla comparativa antes y después

	Antes (E1 y E2)	**Después (E3 y E4)**
Límite de memoria	128 MB	512 MB
Swap del contenedor	384 MB	0 MB
Latencia de /analytics	26 a 438 s	~0,1 s
vmstat durante una llamada	si hasta 21 396, so hasta 65 572, wa 17–42 %	si = 0, so = 0, wa = 0
VmSwap del proceso	83 464 a 101 044 kB	0 kB
memory.current	125 MB de 128 MB	206 MB de 512 MB
Swp del host en htop	creciendo	0K / 2.00G
oom_kill	0	0 (hasta que hubo concurrencia)

Cambios en la VM y limpieza

- Se creó `/swapfile` de 2 GB y una línea en `/etc/fstab` (la VM no tenía swap). Sigue activo.
- `vm.swappiness` quedó en 10 con `sysctl -w` (no persistente).
- El contenedor `reto2_app` queda corriendo con 512 MB y sin swap para la presentación. La limpieza de la guía es `docker rm -f reto2_app`; no la ejecuté para poder mostrar el estado final.
- Los archivos del laboratorio están en `~/laboratorios/lab4` en la VM; los scripts de captura, en `entrega/tools`.

Referencias

[1] A. Silberschatz, P. B. Galvin y G. Gagne, Operating System Concepts, 10a ed. Hoboken, NJ: Wiley, 2018, cap. 10 (memoria virtual y thrashing).

[2] A. S. Tanenbaum y H. Bos, Modern Operating Systems, 4a ed. Boston, MA: Pearson, 2015, cap. 3.

[3] The Linux Kernel documentation, "Control Group v2: Memory", https://docs.kernel.org/admin-guide/cgroup-v2.html.

[4] Docker Docs, "Runtime options with Memory, CPUs, and GPUs: --memory-swap", https://docs.docker.com/config/containers/resource_constraints/.